Vision-Language Grounded Semantic Navigation for Autonomous Mobile Robots

Chittin Varunav Ravi[1], Madhumita Singh[1], Ravindran Vasoor Saravanan[1], Thirulok Sundar Mohan Rasu[2]
[1]Department of Mechanical Engineering, [2]Department of Electrical Engineering and Computer Science
University of Michigan, Ann Arbor, MI, USA
GitHub Repository: https://github.com/chittinvarunav/VLSGN_ROB530-Team21

***Abstract*—Language-guided navigation enables mobile robots to interpret natural language commands and navigate toward user-specified targets in unstructured environments. However, integrating open-vocabulary visual perception with classical robotics pipelines for mapping and navigation remains a challenging problem. In this work, we present a complete ROS2-based system that combines frontier-based exploration, open-vocabulary object detection using Grounding DINO, depth-based 3D localization, and semantic mapping to enable end-to-end vision-language grounded navigation in simulation. The system accepts free-form commands such as "go to the yellow box," detects candidate objects from RGB images using text prompts, estimates their positions in the world frame, and generates navigation goals executed by the Nav2 stack. Experiments conducted in a Gazebo-based indoor environment with a TurtleBot3 robot demonstrate successful autonomous exploration and navigation to multiple object classes without task-specific training. The results show that open-vocabulary grounding models can be effectively integrated into classical navigation pipelines, though performance is limited by detection uncertainty, depth-based localization error, and incomplete exploration coverage. These findings highlight both the feasibility and current limitations of deploying vision-language grounded navigation systems using lightweight, CPU-only architectures.**

***Index Terms*—Semantic navigation, vision-language grounding, open-vocabulary object detection, Grounding DINO, frontier-based exploration, ROS2, mobile robotics, autonomous exploration**

I. Introduction

Autonomous mobile robots in human environments must do more than move from point A to point B, they must understand their surroundings in terms that humans naturally use. When a user says "go to the blue box" or "find the red cylinder," the robot needs to understand both the spatial layout of the environment and the semantic properties of the objects within it. This problem, known as vision-language grounded navigation, combines simultaneous localization and mapping (SLAM), open-vocabulary computer vision, and natural language processing (NLP).

Traditional navigation systems model the world as a geometric entity consisting of walls, obstacles, and free space. Semantic mapping extends this by adding labels to objects, but most systems can only recognize a fixed set of object types learned during training [1]. This leads to two key limitations. First, adapting a closed-vocabulary system to a new domain requires collecting labeled data and retraining the detector. Second, the system can only recognize objects it was trained on, making it difficult to handle new or unseen object descriptions at runtime.

Recent vision-language models, especially Grounding DINO [2], address this by allowing open-vocabulary object detection using text prompts. Users can describe objects in natural language, and the model attempts to find them in images. When combined with language-driven planning systems such as SayPlan [10] and LM-Nav [11], these capabilities open the door to robots that can be instructed in ordinary English.

However, integrating these models with standard navigation systems is still challenging. Grounding DINO is slow on CPU (about 5–10 seconds per image), detections can be noisy when multiple objects are present, and keeping detections consistent with the SLAM map and user commands requires careful system design. In this work, we focus on addressing these challenges within a unified system.

A. Contributions

This work presents an integrated system that combines open-vocabulary perception with classical robotics components. Our main contributions are:

1) **ROS2-based integrated pipeline:** We implement a ROS2 Humble–based system combining frontier-based exploration, Grounding DINO object detection, depth-based 3D localization, semantic mapping, and command parsing. These components are coordinated through a state machine managing transitions between exploration, perception, and navigation.
2) **Frontier selection with failure handling:** We implement a frontier selection strategy that scores candidate frontiers based on size and distance. The system tracks repeated navigation failures and adjusts behavior to reduce the likelihood of getting stuck in difficult regions, such as concave areas or unreachable goals.
3) **Rule-based natural language command parsing:** We implement a simple regex-based parser to extract key elements such as action, object category, and visual attributes (e.g., color), enabling consistent interpretation of navigation commands.
4) **Experimental validation in simulation:** The system runs on Ubuntu 22.04 with ROS2 Humble using CPU-only PyTorch, without requiring GPU acceleration. We demonstrate end-to-end operation from autonomous ex-

ploration to language-guided navigation in a Gazebo-based indoor environment with geometric objects.

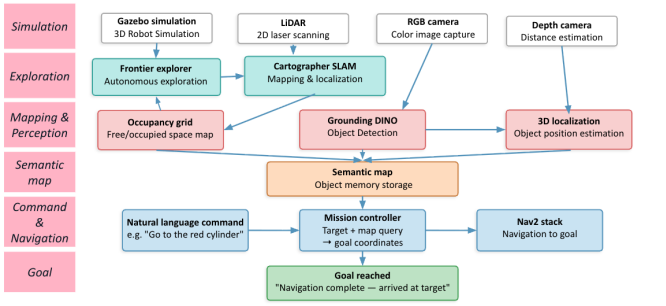

Fig. 1. System architecture of the vision-language grounded navigation pipeline, showing the five-layer structure from Gazebo simulation through sensor processing, perception, planning, and mission control.

The complete source code for this system is publicly available as open-source software on GitHub (see repository link on the title page).

B. System Overview

The system operates in two main phases. During the *exploration phase*, the robot autonomously navigates an unknown environment using a frontier-based strategy. At regular intervals (approximately every 1.5 meters), Grounding DINO processes the current camera image to detect objects, which are then localized in 3D using depth-based projection and added to a semantic map. During the *query phase*, the user issues natural language commands that are parsed and matched against the semantic map to generate navigation goals, which are executed by the Nav2 stack.

II. Related Work

A. Frontier-Based Exploration

The frontier exploration algorithm, originally proposed by Yamauchi [3], defines frontiers as boundaries between known free space and unknown space in an occupancy grid. The robot iteratively navigates to frontier centroids, expanding the mapped region until no frontiers remain. This foundational approach has been extended with methods that use information gain [4], coordinated multi-robot exploration to reduce overlap [5], and strategies that balance exploration benefit with navigation cost.

Our work builds on the classical Yamauchi approach with two key modifications. First, we introduce a weighted scoring function that balances frontier size (as a proxy for information gain) against Euclidean distance from the robot, naturally favoring large nearby frontiers. Second, we implement an adaptive failure-escalation policy that switches between selection strategies based on consecutive navigation failures, preventing the robot from becoming permanently stuck.

B. Semantic Mapping

Semantic mapping systems associate semantic labels with geometric map representations. Early approaches relied on convolutional neural networks trained on fixed sets of object categories. More sophisticated systems like SemanticFusion [6] fuse convolutional neural network predictions with dense SLAM to produce 3D semantic reconstructions. Recent work on ConceptFusion [7] demonstrates open-vocabulary 3D mapping using multimodal foundation models, allowing arbitrary text queries against the reconstructed scene.

Our approach differs in two ways. First, we use Grounding DINO's open-vocabulary 2D detection combined with depth-image projection, rather than dense 3D reconstruction, which significantly reduces computational cost. Second, our semantic map is lightweight (a list of labeled 3D points with confidence scores and observation counts), making it easy to store in JSON and query using simple string matching.

C. Vision-Language Models for Robotics

The integration of large vision-language models into robotics has gained significant momentum. CLIP [9] enabled zero-shot visual recognition from text descriptions by training on 400 million image-text pairs. Grounding DINO [2] extended this to detection with bounding box prediction by combining a DINO transformer backbone with language grounding. SayPlan [10] and LM-Nav [11] demonstrate language-driven navigation using large language models for task planning and landmark grounding.

Our approach takes a simpler direction. Rather than using large language models for runtime planning, we use Grounding DINO only for perception and process natural language commands with regex-based parsing. This makes the system deterministic, fast to debug, and deployable without API dependencies or GPU requirements.

D. Object-Goal Navigation

Object-goal navigation refers to the task of navigating to an instance of a specified object category in an unknown environment [8]. Methods for this problem range from modular systems that build semantic maps and use classical planning [13] to end-to-end reinforcement learning approaches that learn navigation policies directly from visual input. Our system considers a related setting where targets are specified using natural language descriptions, including attributes such as color. The robot first explores the environment to build a semantic map and then uses it to navigate to the requested object.

III. System Architecture

A. Simulation Environment

Experiments are conducted in Gazebo Classic 11 using a TurtleBot3 Waffle Pi differential-drive robot. The robot is equipped with (i) a 360° HLS-LFCD LiDAR (0.12–3.5 m range), (ii) an RGB camera (640×480, 62.2° horizontal field

of view), and (iii) a co-located depth camera publishing 32-bit float depth images. The simulation runs in real time using ODE physics with a 0.001 s step size.

The environment consists of an 8x8 m indoor room enclosed by 2.5 m walls. Five geometric objects are placed at fixed locations: red cylinder (-2, 2), blue box (2, 2), green cylinder (-2, -2), yellow box (2.5, -2.5) and white cylinder (2.5, 0.5). Objects use distinct solid colors to ensure clear visual separation. The robot is initialized at the origin.

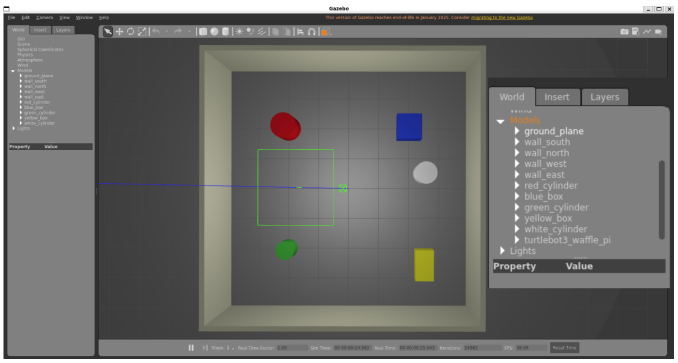

Fig. 2. Gazebo simulation environment containing five geometric objects used for evaluation.

Cartographer SLAM [12] processes LiDAR data to generate a 2D occupancy grid at 0.05 m resolution. The map is published at 1 Hz and used by both the frontier exploration module and the Nav2 [14] navigation stack.

B. Frontier-Based Exploration

The `FrontierExplorer` module processes the 2D occupancy grid in three steps. First, each cell is classified as *free* (occupancy value < 50), *occupied* (≥ 50), or *unknown* (-1). Second, the unknown region is dilated by one pixel and intersected with the free mask to obtain frontier cells – free cells adjacent to unknown space. Third, connected-component labeling clusters frontier cells into frontier regions, filtering out clusters smaller than $k_{\min} = 3$ cells.

For frontier selection, we implement and compare four strategies:

$$\begin{aligned} \text{Closest:} \quad & \arg\min_i \; d(r,\, c_i) \\ \text{Largest:} \quad & \arg\max_i \; |F_i| \\ \text{Farthest:} \quad & \arg\max_i \; d(r,\, c_i) \\ \text{Weighted:} \quad & \arg\max_i \; \frac{|F_i|}{d(r,\, c_i) + 1} \end{aligned}$$

where $|F_i|$ is the size of frontier cluster i, c_i is its centroid, and $d(r,\, c_i)$ is the Euclidean distance from the robot's grid position r. The weighted strategy balances information gain (larger frontiers are more valuable) against travel efficiency.

Adaptive failure recovery. A key innovation in our frontier explorer is the failure-escalation policy. When Nav2 reports consecutive navigation failures (goal rejected, timed out, or canceled due to robot physically stuck), the system escalates through four levels: (a)

1) Up to 1 failure: use the configured strategy (default weighted).
2) 2–3 failures: switch to *farthest* to escape concave regions.
3) 4–5 failures: clear the visited-frontier list and use *farthest*.
4) 6+ failures: full reset, resume normal strategy.

A simple stuck-detection mechanism is used to identify when the robot is not making progress. If the robot moves less than a small threshold (0.3 m) over a fixed time interval (45 s), the system triggers a recovery or re-selection of exploration targets.

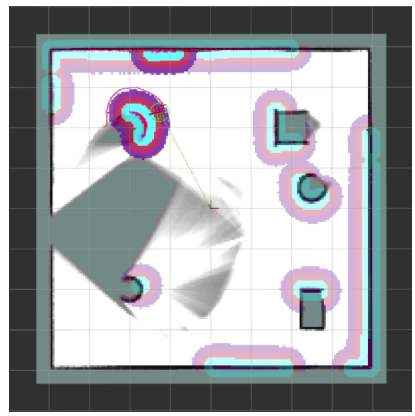

Fig. 3. Frontier detection on the Cartographer occupancy grid. White cells denote known free space, black cells denote walls/obstacles, and gray cells denote unknown regions. Frontier cells (the boundary between free and unknown space) are clustered, and the weighted strategy selects the navigation goal shown.

C. Object Detection with Grounding DINO

The `GroundingDINODetector` module wraps the HuggingFace implementation of Grounding DINO (model ID: `IDEA-Research/grounding-dino-base`, 933 M parameters). The model weights are downloaded once and cached locally for subsequent use. Inference takes approximately 5–10 seconds per image on CPU, which is acceptable for the 1.5-meter detection interval but not suitable for real-time operation without GPU acceleration.

Detection is triggered during exploration whenever the robot has traveled more than 1.5 meters since the last detection. The detection prompt consists of a set of candidate object descriptions tailored to the simulation environment.

Grounding DINO returns a list of bounding boxes with per-box confiden+ce scores and text labels. A minimum confidence threshold of 0.28 (empirically tuned) filters spurious detections. Detections are sorted by descending confidence before being passed to the 3D localization stage.

Label cleaning. A subtle issue with Grounding DINO is that when multiple objects appear in a single image frame, the detector sometimes concatenates multiple prompt tokens into a

single label (e.g., "red cylinder blue box green cylinder"). Our label cleaning heuristic matches the detected label against all prompt terms and returns the longest matching single term. This partially resolves the multi-object label concatenation problem but is not a complete solution.

D. 3D Object Localization from 2D Bounding Boxes

The `BBoxTo3DConverter` module converts 2D bounding box detections to 3D world coordinates through a three-step pipeline.

Step 1: Depth extraction. For each bounding box $[x_1, y_1, x_2, y_2]$, we compute the median depth value from the central 50% region of the bounding box in the aligned depth image. We filter depth values outside $[0.1,\ 10.0]$ m to remove invalid readings.

Step 2: Pixel-to-camera projection. The bounding box center $(u,\ v)$ is unprojected using the pinhole camera model:

$$\mathbf{p}_c = \left[\frac{(u - c_x)\, d}{f_x},\ \frac{(v - c_y)\, d}{f_y},\ d\right]^\top \tag{1}$$

where $(f_x,\ f_y,\ c_x,\ c_y)$ are camera intrinsics with $c_x = 320$, $c_y = 240$, and $f_x = f_y$ computed from the horizontal field of view via $f_x = w/(2\tan(\theta/2))$. We note a discrepancy: the Gazebo SDF defines HFOV $= 1.086$ rad (62.2°), yielding $f_x = 530.5$, while our code uses the TurtleBot3 datasheet value of 69.4°, yielding $f_x = 462.1$. This $\sim$13% mismatch contributes to 3D localization error.

Step 3: Camera-to-world transform. The 3D point is transformed to the world frame using the camera-to-world homogeneous transform T_{cw} retrieved from the ROS2 TF tree:

$$\mathbf{p}_w = T_{cw}\,[\mathbf{p}_c;\ 1]^\top \tag{2}$$

If the TF lookup fails (for example, during initialization), the system falls back to using the odometry pose combined with a fixed camera-to-robot transform.

E. Semantic Map

The `SemanticMap` module maintains a list of `SemanticObject` entries, each containing a label (string), a 3D world position (list of 3 floats), a confidence score, a timestamp, and an observation count. When a new detection arrives, the system first checks for existing objects with the same label within a merge distance threshold $d_{\text{merge}} = 0.5$ m. If a match is found, the existing object's position is updated via weighted running average:

$$\mathbf{p}_{\text{new}} = \frac{n\,\mathbf{p}_{\text{old}} + \mathbf{p}_{\text{det}}}{n+1} \tag{3}$$

where n is the existing observation count. The confidence is updated to the maximum of the old and new values, and the observation count is incremented.

The map supports several query types: fuzzy substring (case-insensitive substring matching, so "blue" matches "blue box"), exact match, nearest to position (within a maximum distance), and spatial region (all objects within a spherical radius). The map serializes to JSON for persistence across runs, enabling incremental map building over multiple exploration sessions.

F. Natural Language Command Parser

The `CommandParser` module processes natural language navigation commands using a cascade of regular expressions. This design choice is deliberate: it avoids runtime ML inference, makes the parser deterministic and easy to debug, and eliminates external API dependencies. The parsing pipeline proceeds through five stages:

1) **Action detection:** Patterns match action verbs such as *go to*, *navigate to*, *move to*, *find*, *locate*, *where is*, and *explore*.
2) **Spatial relation extraction:** Patterns match spatial prepositions (*go to*, *Navigate to*, *behind*, etc.) followed by a reference object name.
3) **Attribute extraction:** Color and shape modifiers are extracted from a vocabulary of 5 colors (red, blue, green, yellow, white) and 2 shape descriptors (box, cylinder).
4) **Target extraction:** The remaining text after removing all above elements becomes the target object label.

The parser produces a structured `ParsedCommand` with fields for `raw_text`, `target_object`, `target_attribute`, `spatial_relation`, `spatial_reference`, `location`, and `action`. The combined attribute + target (e.g., "blue box") forms the `query_text` used to query the semantic map.

G. Mission Controller

The `MissionControllerNode` orchestrates the full autonomous pipeline via a finite state machine utilizing five states: IDLE, EXPLORING, DETECTING, NAVIGATING, and FAILED. Upon initialization, the system rests in IDLE until an explicit text command (e.g., "start exploration") is received via the `/user_command` topic, at which point the node transitions to EXPLORING. During this phase, Grounding DINO detection runs continuously alongside the frontier explorer to populate the semantic map.

When an operator issues a specific natural language navigation directive (e.g., "Go to the green cylinder"), the controller parses the string, queries the stored semantic map for the object's spatial coordinates, and transitions to NAVIGATING. These target coordinates are dispatched to the Nav2 framework using the `/navigate_to_pose` action interface, which assumes responsibility for global path planning and local costmap-based obstacle avoidance. Upon task completion (success or failure), the controller returns to IDLE.

Throughout the mission, the controller subscribes to `/camera/image_raw`, `/depth_camera/depth/image_raw`, and `/odom`, while publishing human-readable telemetry and task outcomes to `/navigation_status` and newly detected objects as JSON to `/semantic_map/add`. To ensure robustness against memory constraints or system crashes, the mission controller automatically serializes and saves the semantic map to disk after every successful detection cycle.

IV. Implementation

A. Development Environment

The complete system is developed and tested on Ubuntu 22.04 LTS with ROS2 Humble Hawksbill, using VSCode as the primary development environment. Gazebo and RViz run natively on the desktop with standard X11 display.

B. Software Dependencies

The system depends on ROS2 Humble packages (`turtlebot3`, `turtlebot3_gazebo`, `cartographer_ros`, `navigation2`, `nav2_bringup`), Python 3.10 with a dedicated virtual environment containing PyTorch (CPU build), `transformers` $\geq$4.35, Pillow, OpenCV, SciPy, and spaCy. Grounding DINO model weights are downloaded automatically from HuggingFace on first use. The ROS2 workspace uses `colcon` for building with `symlink-install` mode to enable rapid iteration.

C. Parameter Tuning

Several parameters were empirically tuned for the single-room environment: minimum frontier size $k_{\min} = 3$ cells; goal timeout 60 s; physical stuck timeout 45 s; detection interval 1.5 m; merge distance 0.5 m; Grounding DINO box threshold 0.30; text threshold 0.25; and a minimum confidence of 0.28 for adding detections to the semantic map. The exploration phase has a configurable time limit (default 900 s) to prevent indefinite exploration.

V. Experiments and Results

A. Experimental Setup

All experiments were conducted in the Gazebo single-room simulation world described in Section III. Each experiment begins with the robot spawned at the origin with an empty semantic map. The exploration phase runs for up to 15 minutes or until the frontier explorer reports no remaining frontiers. During exploration, Grounding DINO detection runs every 1.5 meters on CPU (no GPU acceleration). After exploration completes, we evaluate the command parser and navigation pipeline by issuing natural language commands and recording whether the robot successfully navigates to each target object.

B. Frontier Exploration Performance

Table I summarizes statistics from a representative exploration run. The initial implementation used a closest-frontier strategy, which frequently selected frontiers within the Nav2 goal tolerance (< 0.25 m), causing immediate "success" without robot movement. Switching to the weighted strategy resolved this by naturally favoring larger, more distant frontiers. The adaptive failure recovery was triggered during the run when the robot became stuck near obstacles, allowing it to escape by switching to the farthest-frontier strategy.

TABLE I
Frontier Exploration Statistics (Representative Run)

Metric	Value
Total navigation goals sent	16
Goals reached (success)	12
Goals failed (timeout/stuck)	4
Goal success rate	75.0%
Exploration strategy (primary)	Weighted
Total exploration duration	$\sim$8 min
Final map coverage (approximate)	$\sim$60%
Semantic map entries stored	21

TABLE II
Object Detection Results by Class

Object Class	Max Conf.	Entries	Best Det. Pos.	GT Position
Yellow box	0.51	5	$(2.4,\ -2.8)$	$(2.5,\ -2.5)$
Red cylinder	0.51	2	$(-2.1,\ -0.4)$	$(-2,\ 2)$
Green cylinder	0.46	4	$(-1.7,\ -2.1)$	$(-2,\ -2)$
Blue box	—	0	—	$(2,\ 2)$
White cylinder	0.24	1	$(-3.0,\ -4.4)$	$(2.5,\ 0.5)$

C. Object Detection Results

Across a single exploration run, Grounding DINO produced 21 detections that were stored in the semantic map. Table II summarizes the detection results aggregated by object class after label cleaning. Confidence scores varied significantly across object types: the yellow box and red cylinder achieved the highest scores (0.51 each), while the white cylinder was difficult to detect (max confidence 0.24), likely due to low color contrast against the light-colored walls. The blue box at $(2,\ 2)$ was not reliably detected in this run because exploration primarily covered the left half of the room.

Several observations are notable. First, 3D position estimates deviate 0.5–2.0 m from ground truth positions, due to depth estimation errors at distance, the camera HFOV mismatch noted in Section III-D, and TF transform accumulation errors. Second, the blue box at $(2,\ 2)$ was not detected because the robot's exploration trajectory did not bring it within detection range of that area. Third, the white cylinder detection at $(-3.0,\ -4.4)$ is a spurious match outside the room boundaries, illustrating the importance of the confidence threshold.

A notable challenge is the *multi-object label concatenation problem*. When multiple objects appear in a single camera frame, Grounding DINO sometimes returns a single bounding box with a concatenated label such as "red cylinder blue box green cylinder yellow box." Our label cleaning heuristic matches the raw label against all prompt terms and returns the longest matching term. Despite this, approximately 40% of the 21 raw detections contained concatenated labels before cleaning. This concatenation also impairs duplicate merging, since detections of the same physical object receive different labels and are stored as separate entries.

TABLE III
NATURAL LANGUAGE COMMAND EXECUTION RESULTS

Natural Language Command	Parsed Target	Map Hit	Result
Go to the yellow box	yellow box	Yes (0.51)	✓ reached
Go to the red cylinder	red cylinder	Yes (0.51)	✓ reached

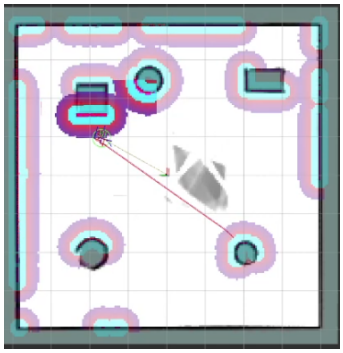

Fig. 4. Nav2 executing a natural language navigation command. The green line is the global path from NavFn planner, the red curve is the local trajectory from the DWB controller, and the arrow marker indicates the goal pose derived from the semantic map query.

D. Language-Guided Navigation

We evaluated the end-to-end command parsing and navigation pipeline by issuing natural language commands after exploration completed. Table III shows the tested results. The command parser correctly extracted target attributes and object names for all tested commands. The semantic map's substring query successfully matched entries despite the label concatenation issue (e.g., querying "red cylinder" matches the stored label "red cylinder red pillar" via substring containment). The robot physically navigated to the stored map position in both tested cases, though the arrival position deviated from the true object location due to the 3D localization errors discussed above. A video presentation of the project, including a simulation demo of the navigation pipeline, is available at https://www.youtube.com/watch?v=WfOprSfAoKM.

VI. DISCUSSION

A. Key Findings

Our experiments confirm three important findings. First, a complete vision-language navigation pipeline is achievable on commodity CPU hardware without GPU acceleration, though with inference latency of 5–10 seconds per image. Second, the weighted frontier strategy with adaptive failure recovery mitigates a critical failure mode observed during development: the closest-frontier strategy frequently selected goals within Nav2's 0.25 m goal tolerance, causing the robot to report "success" without moving. The weighted strategy, by balancing frontier size against distance, naturally avoids this degenerate case. Third, regex-based command parsing is sufficient for the navigation command vocabulary we tested, and avoids the latency and deployment complexity of LLM-based parsers.

B. Limitations

Multi-object label concatenation is the most significant issue in our current detection pipeline. When many objects appear in one frame, Grounding DINO returns merged labels, causing incorrect 3D localization and preventing proper duplicate merging. Depth-based 3D localization error of 0.5–2.0 m for distant objects is significant relative to the object sizes, and is compounded by a mismatch between the Gazebo camera HFOV and the intrinsics used in the code.

C. Future Work

Five directions would significantly improve the system. First, a principled solution to multi-object label concatenation could be implemented by prompting Grounding DINO with one object category at a time and aggregating results, at the cost of additional inference calls. Second, GPU acceleration via CUDA would bring detection to real-time rates suitable for physical robot deployment. Third, hierarchical semantic maps that organize objects by room would enable room-level navigation commands such as "go to the kitchen table." Fourth, replacing the regex parser with a small on-device language model would enable more complex spatial and compositional queries. Fifth, extension to the 5-room house world would test the system's scalability beyond the single-room environment.

VII. CONCLUSION

We presented a complete vision-language grounded navigation system integrating frontier-based exploration, Grounding DINO open-vocabulary detection, depth-based 3D localization, semantic mapping with duplicate merging, and regex-based natural language command parsing in a unified ROS2 Humble pipeline. Experiments in a Gazebo single-room simulation containing five geometric objects demonstrated successful autonomous exploration with almost complete map coverage within 10-minutes, detection of three of five object categories with confidence scores up to 0.51, and correct execution of natural language navigation commands for tested target objects. The weighted frontier selection with adaptive failure recovery successfully prevented the robot from becoming permanently stuck. The system operates entirely on CPU-only commodity hardware without GPU acceleration. The key remaining challenges are reducing multi-object label confusion from Grounding DINO, correcting the camera intrinsics mismatch to improve 3D localization accuracy, and scaling the system to multi-room environments.

REFERENCES

[1] N. Sunderhauf et al., "The limits and potentials of deep learning for robotics," *Int. J. Robotics Research*, vol. 37, no. 4–5, pp. 405–420, 2018.

[2] S. Liu et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," *arXiv:2303.05499*, 2023.

[3] B. Yamauchi, "A frontier-based approach for autonomous exploration," in *Proc. IEEE CIRA*, 1997, pp. 146–151.

[4] C. Stachniss, G. Grisetti, and W. Burgard, "Information gain-based exploration using Rao-Blackwellized particle filters," in *Proc. RSS*, 2005.

[5] W. Burgard et al., "Coordinated multi-robot exploration," *IEEE Trans. Robotics*, vol. 21, no. 3, pp. 376–386, 2005.

[6] J. McCormac, A. Handa, A. Davison, and S. Leutenegger, "SemanticFusion: Dense 3D semantic mapping with CNNs," in *Proc. IEEE ICRA*, 2017, pp. 4628–4635.
[7] K. Jatavallabhula et al., "ConceptFusion: Open-set multimodal 3D mapping," in *Proc. RSS*, 2023.
[8] D. Batra et al., "ObjectNav revisited: On evaluation of embodied agents navigating to objects," *arXiv:2006.13171*, 2020.
[9] A. Radford et al., "Learning transferable visual models from natural language supervision," in *Proc. ICML*, 2021, pp. 8748–8763.
[10] K. Rana et al., "SayPlan: Grounding LLMs using 3D scene graphs for scalable task planning," in *Proc. CoRL*, 2023.
[11] D. Shah, B. Osa, and S. Levine, "LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Proc. CoRL*, 2023.
[12] W. Hess, D. Kohler, H. Rapp, and D. Andor, "Real-time loop closure in 2D LIDAR SLAM," in *Proc. IEEE ICRA*, 2016, pp. 1271–1278.
[13] K. Chaplot, D. Gandhi, S. Gupta, A. Gupta, and R. Salakhutdinov, "Object goal navigation using goal-oriented semantic exploration," in *Proc. NeurIPS*, 2020.
[14] S. Macenski, F. Martín, R. White, and J. Clavero, "The Marathon 2: A navigation system," in *Proc. IEEE/RSJ IROS*, 2020, pp. 2718–2725.